

QuizItOut: Mentor Plan (Doc-to-Quiz CI/CD Project)

Oct 3, 2026 · @shweta

1. Reality check (pehle ye padho)

DocQuiz ek aisa app hai jisme tum PDF, DOCX, PPTX ya TXT upload karti ho, app text nikalta hai, chunks banata hai, local LLM (Ollama) se MCQs generate karta hai, har MCQ ko validate karta hai (schema + grounding check), aur browser mein quiz dikhata hai. Uske peeche ek Jenkins pipeline hai: lint, test, quality gate, Docker build, Trivy scan, push, deploy, smoke test, rollback.

"Production-level all-in-one" kitna sach hai? Almost, par poora nahi. Resume mein wahi likho jo sach hai.

Area	Is project mein?
Git/GitHub version control	Yes
Automated tests + coverage gate	Yes
Multi-stage Docker build	Yes
Vulnerability scan (Trivy)	Yes
Registry + SHA-tagged images	Yes
Auto deploy + health check + rollback	Yes
Monitoring / metrics / alerts	No (stretch: <code>/metrics</code> + Prometheus)
Infrastructure as Code (Terraform)	No
Kubernetes / orchestration	No
Secrets manager, auth, rate limiting	No
Staging + prod environments	No

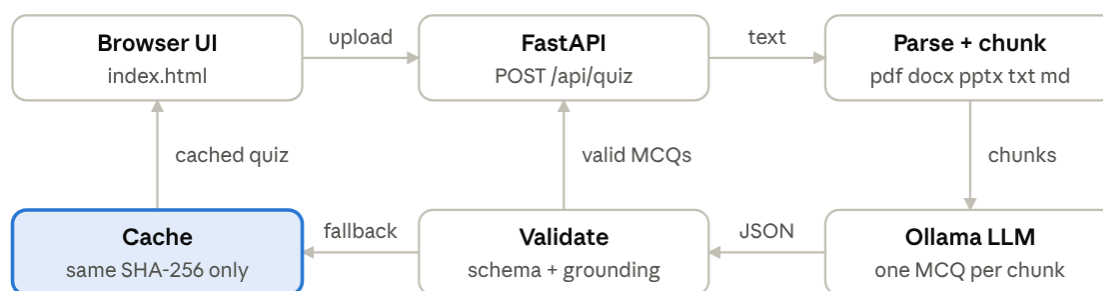
Resume line: **"End-to-end CI/CD pipeline with automated testing, vulnerability scanning and automated rollback"**. "Production-level" mat likhna, interviewer ek hi sawal se pakad lega.

Demo fallback (design decision). Demo ke din LLM kabhi bhi ladkhada sakta hai. Isliye: demo PDF ka quiz real pipeline se ek baar generate karo, haath se review karo, `data/demo/demo_quiz.json` mein save karo. App is JSON ko **cache** ki tarah tabhi serve karta hai jab (a) LLM fail ho ya koi valid question na nikle, aur (b) uploaded file ka SHA-256 demo file se match kare. Response mein `source: "live"` ya `"cached"` aata hai aur UI badge dikhata hai. Ye graceful degradation hai. Cache ko live RAG output bolke dikhana galat hoga, isliye demo mein seedha bolo: "live generation hai, cache sirf safety net hai."

□ 2. Architecture (poora picture)

Do alag duniya hain: **app** (upload se quiz tak) aur **pipeline** (code push se deploy tak). Pehle app ka flow, phir pipeline ke stages.

A quiz is generated live; a hash-matched cache catches failures



app request flow · 6 components, 1 fallback path

Live generation default hai. Validator kuch valid na de ya Ollama down ho, tab cache dekha jaata hai, aur wo sirf tab lagta hai jab file ka SHA-256 demo file se match kare.

`PREFER_CACHE=true` pe cache pehle dekha jaata hai.

CI/CD pipeline (Jenkins stages, order mein)

1. **Checkout:** GitHub se code, git SHA se `TAG` banta hai.
2. **Build test image:** Docker `test` target, models baked.
3. **Lint:** ruff.
4. **Unit + API tests:** pytest, coverage 80% se kam = fail.
5. **Quality gate:** golden document pe retrieval hit-rate@3 ≥ 0.8 .
6. **Build runtime image:** non-root, SHA tag.
7. **Trivy scan:** HIGH/CRITICAL fixable CVE = fail.
8. **Push:** Docker Hub pe `:sha` aur `:latest`.
9. **Deploy:** `docker compose up` us SHA ke saath.

10. **Smoke test:** container healthy aur `/api/quiz/demo` 200, warna **rollback** purane SHA pe.

Koi bhi stage fail hua toh pipeline wahin ruk jaati hai aur kharab code aage nahi jaata.

□ 3. Folder structure (har cheez kyun hai)

Rule: **ek folder = ek zimmedari**. Isse testing aur viva dono aasan hote hain.

```
docquiz/
├── app/                                # sab runtime code
│   ├── __init__.py
│   ├── main.py                        # FastAPI routes + wiring (patla, logic yahan
nahi)
│   ├── config.py                     # env vars ek jagah
│   ├── schemas.py                   # Pydantic models = data ka contract
│   ├── parsers.py                   # file -> plain text
│   ├── rag/
│   │   ├── __init__.py
│   │   ├── chunker.py               # text -> chunks, chunk sampling
│   │   └── retriever.py             # hybrid search + rerank (topic mode)
│   ├── quiz/
│   │   ├── __init__.py
│   │   └── llm.py                   # Ollama client (sirf yahi network call karta
hai)
│   │   ├── generator.py             # chunk -> MCQ loop + retry
│   │   ├── validator.py             # schema + grounding check
│   │   └── fallback.py              # SHA-256 keyed cache
│   └── static/index.html             # simple quiz UI
├── data/demo/                        # demo_doc.pdf + demo_quiz.json (reviewed
cache)
├── scripts/
│   ├── __init__.py
│   ├── build_demo_cache.py          # cache ek baar generate karne ki script
│   └── quality_gate.py              # retrieval hit-rate gate (CI mein)
├── tests/
│   ├── __init__.py
│   ├── samples.py                   # shared fake LLM + sample data
│   └── golden/                      # golden.json + notes.txt (quality gate ka
data)
│   └── test_*.py
├── Dockerfile                        # multi-stage: base / test / runtime
├── docker-compose.yml
├── Jenkinsfile                       # pipeline as code
└── requirements.txt                  # runtime deps
```

```
├─ requirements-dev.txt      # pytest, coverage, ruff
├─ pytest.ini
├─ .dockerignore    .gitignore
└─ README.md
```

Kyun aise?

- `llm.py` akela network call karta hai. Baaki sab pure functions hain, toh tests mein bas ek `FakeLLM` pass karna hota hai. Ye **dependency injection** hai, aur viva ka achha topic.
- `main.py` patla hai: request lo, functions bulao, response do. Business logic routes mein ghusa ho toh test karna mushkil hota hai.
- `schemas.py` alag hai kyunki LLM aur API dono ka output ek hi contract se validate hota hai.
- `data/demo/` mein cache aur demo PDF saath rehte hain, taaki SHA-256 match ho sake.
- `tests/` aur `scripts/` ko Docker ke `test` stage mein hi copy karenge, runtime image mein nahi. Production image chhoti rehti hai.

□ 4. Theory pehle (code se pehle ye samjho)

4.1 Parsing

PDF/DOCX/PPTX sab ke andar text alag tareeke se store hota hai. Parser ka kaam ek hi hai: **kisi bhi format se plain text nikalna**. `pypdf` PDF pages se, `python-docx` paragraphs se, `python-pptx` har slide ke text frames se. Limitation: scanned PDF (image) mein text nahi hota, uske liye OCR chahiye. Hum us case ko saaf error dete hain, OCR scope se bahar hai.

4.2 Chunking

LLM ko poora document nahi de sakte (context limit, slow, quality girti hai). Toh text ko **chunks** mein todte hain, ~800 characters, paragraph boundary pe. **Overlap** (100 chars) isliye ki ek idea do chunks ke beech kat na jaye. Chhota chunk = precise par context kam. Bada chunk = context zyada par noisy. 500 se 1000 chars ek achha start hai.

4.3 Quiz generation retrieval nahi hai

RAG mein sawal aata hai aur hum uske jawab ke chunks dhundhte hain. Quiz mein koi sawal nahi hota, **poora document cover karna hota hai**. Isliye default mode mein hum chunks ko document mein barabar doori pe **sample** karte hain (`pick_spread`) aur har chunk se ek MCQ banate hain. Retrieval tab kaam aata hai jab user bole "sirf photosynthesis pe quiz" (**topic mode**).

4.4 Hybrid search + rerank (topic mode)

- **Dense (embeddings + FAISS):** `all-MiniLM-L6-v2` har chunk ko 384-number ke vector mein badalta hai. Matlab pakadta hai, exact words nahi. "car" aur "automobile" paas aate hain.
- **Sparse (BM25):** keyword matching. Rare words aur exact terms (naam, acronyms) mein dense se behtar.
- **Fusion (RRF, Reciprocal Rank Fusion):** dono ki ranked lists ko score $1 / (60 + \text{rank})$ se jodte hain. Jo chunk dono mein upar ho wo jeetta hai. Scores alag scale ke hote hain, isliye rank use karte hain, score nahi.
- **Rerank (cross-encoder):** top ~15 candidates ko `ms-marco-MiniLM` ke saath (query, chunk) joda banake dobara score karte hain. Ye slow par accurate hai, isliye sirf shortlist pe chalte hain. Pattern: **fast retrieval se shortlist, slow reranker se final.**

4.5 LLM se structured output

Ollama mein `format: "json"` dene se output valid JSON hone ki koshish karta hai. Phir bhi galat keys, 3 options, ya `answer_index` 9 aa sakta hai. **LLM ka output untrusted input hai**, user input ki tarah. Isliye validate karte hain. `temperature` 0.3 rakhte hain taaki output stable rahe.

4.6 Validation: schema + grounding

1. **Schema (Pydantic):** question string, exactly 4 distinct options, `answer_index` 0 se 3.
2. **Grounding:** LLM ko bolte hain ki jawab ka saboot ek **exact sentence** `source_snippet` mein copy kare. Hum check karte hain ki wo sentence chunk mein sach mein hai. Nahi hai toh question reject.

Honest limitation: grounding ye prove karta hai ki LLM ne source se quote kiya, ye nahi ki `answer_index` sahi option pe hai. Isliye demo cache haath se review hota hai. Viva mein ye limitation khud bolna achha impression deta hai.

4.7 Cache by content hash

SHA-256 file ke bytes ka fingerprint hai. Same file = same hash, ek bhi byte badla toh alag. Isliye cache sirf asli demo file pe lagta hai, kisi aur document pe nahi.

4.8 Security: LLM output ko UI mein daalna

LLM ka text `innerHTML` mein daalna XSS hai. Hum frontend mein `textContent` use karte hain, jo text ko text hi rakhta hai.

❑ 5. Backend code, file by file

Har file: pehle **kyun**, phir code. Sab `docquiz/` ke andar, Python 3.11.

5.1 `app/config.py` : saari settings ek jagah

Env vars se settings padhte hain taaki Docker/Jenkins mein code badle bina behaviour badal sake (12-factor app ka idea). `frozen=True` matlab ban jaane ke baad badal nahi sakti.

```
import os
from dataclasses import dataclass

@dataclass(frozen=True)
class Settings:
    ollama_url: str = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")
    model: str = os.getenv("OLLAMA_MODEL", "llama3")
    chunk_size: int = int(os.getenv("CHUNK_SIZE", "800"))
    chunk_overlap: int = int(os.getenv("CHUNK_OVERLAP", "100"))
    max_questions: int = int(os.getenv("MAX_QUESTIONS", "5"))
    llm_timeout: int = int(os.getenv("LLM_TIMEOUT", "120"))
    max_upload_mb: int = int(os.getenv("MAX_UPLOAD_MB", "10"))
    demo_quiz_path: str = os.getenv("DEMO_QUIZ_PATH",
    "data/demo/demo_quiz.json")
    prefer_cache: bool = os.getenv("PREFER_CACHE", "false").lower() == "true"

settings = Settings()
```

5.2 `app/schemas.py` : data ka contract

Pydantic class ek **validator** bhi hai. LLM ka dict seedha `MCQ(**raw)` mein daalo, galat hai toh exception aata hai. `QuizResponse.source` batata hai ki quiz live hai ya cached.

```
from pydantic import BaseModel, field_validator, model_validator

class MCQ(BaseModel):
    question: str
    options: list[str]
    answer_index: int
    source_snippet: str

    @field_validator("options")
```

```

    @classmethod
    def four_distinct_options(cls, v: list[str]) -> list[str]:
        if len(v) != 4 or len({o.strip().lower() for o in v}) != 4:
            raise ValueError("need exactly 4 distinct options")
        return v

    @model_validator(mode="after")
    def index_in_range(self):
        if not 0 <= self.answer_index < 4:
            raise ValueError("answer_index must be 0-3")
        return self

class QuizResponse(BaseModel):
    source: str # "live" or "cached"
    filename: str
    questions: list[MCQ]

```

5.3 app/parsers.py : file se text

Ek dictionary extension se function map karti hai (`PARSERS`). Naya format add karna = ek function + ek entry. Khaali text aaye toh saaf error, taaki scanned PDF pe user ko pata chale.

```

from pathlib import Path

from docx import Document
from pptx import Presentation
from pypdf import PdfReader

def parse_pdf(path: str) -> str:
    reader = PdfReader(path)
    return "\n\n".join((page.extract_text() or "") for page in reader.pages)

def parse_docx(path: str) -> str:
    doc = Document(path)
    return "\n\n".join(p.text for p in doc.paragraphs if p.text.strip())

def parse_pptx(path: str) -> str:
    out = []
    for slide in Presentation(path).slides:
        for shape in slide.shapes:
            if shape.has_text_frame and shape.text_frame.text.strip():

```

```

        out.append(shape.text_frame.text)
    return "\n\n".join(out)

def parse_text(path: str) -> str:
    return Path(path).read_text(encoding="utf-8", errors="ignore")

PARSERS = {
    ".pdf": parse_pdf,
    ".docx": parse_docx,
    ".pptx": parse_pptx,
    ".txt": parse_text,
    ".md": parse_text,
}

def parse_file(path: str) -> str:
    ext = Path(path).suffix.lower()
    if ext not in PARSERS:
        raise ValueError(f"Unsupported file type: {ext}")
    text = PARSERS[ext](path)
    if not text.strip():
        raise ValueError("No extractable text (scanned PDF?)")
    return text

```

5.4 app/rag/chunker.py : chunking + sampling

Paragraph boundary pe todta hai, ek chunk `size` se bada nahi hota. Bahut lamba paragraph ho toh seedha kaat deta hai. `pick_spread` chunks ko document mein barabar doori pe chunta hai taaki quiz poore document ko cover kare.

```

import re

def chunk_text(text: str, size: int = 800, overlap: int = 100) -> list[str]:
    paragraphs = [p.strip() for p in re.split(r"\n\s*\n", text) if p.strip()]
    chunks: list[str] = []
    current = ""
    for para in paragraphs:
        if len(current) + len(para) + 2 <= size:
            current = f"{current}\n\n{para}".strip()
            continue
        if current:
            chunks.append(current)
            tail = current[-overlap:] if (current and overlap) else ""

```

```

        candidate = f"{tail} {para}".strip()
        current = candidate if len(candidate) <= size else para
        while len(current) > size: # one huge paragraph
            chunks.append(current[:size])
            current = current[size - overlap:]
    if current:
        chunks.append(current)
    return chunks

def pick_spread(chunks: list[str], n: int) -> list[str]:
    if len(chunks) <= n:
        return chunks
    step = len(chunks) / n
    return [chunks[int(i * step)] for i in range(n)]

```

5.5 app/rag/retriever.py: hybrid search + rerank (topic mode)

Models **lazy-load** hote hain (pehli baar use pe), taaki app start hone mein aur tests mein import slow na ho. `rrf` ek pure function hai, isliye alag se test ho sakta hai.

```

import faiss
import numpy as np
from rank_bm25 import BM25Okapi
from sentence_transformers import CrossEncoder, SentenceTransformer

_embedder = None
_reranker = None

def _get_embedder():
    global _embedder
    if _embedder is None:
        _embedder = SentenceTransformer("all-MiniLM-L6-v2")
    return _embedder

def _get_reranker():
    global _reranker
    if _reranker is None:
        _reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
    return _reranker

def rrf(rankings: list[list[int]], k: int = 60) -> list[int]:
    """Reciprocal Rank Fusion: merge several ranked id lists."""

```

```

score: dict[int, float] = {}
for ranking in rankings:
    for rank, doc_id in enumerate(ranking):
        score[int(doc_id)] = score.get(int(doc_id), 0.0) + 1.0 / (k +
rank + 1)
    return sorted(score, key=score.get, reverse=True)

class HybridRetriever:
    def __init__(self, chunks: list[str]):
        self.chunks = chunks
        vecs = _get_embedder().encode(chunks, normalize_embeddings=True)
        self.index = faiss.IndexFlatIP(vecs.shape[1]) # inner product =
cosine (normalized)
        self.index.add(np.asarray(vecs, dtype="float32"))
        self.bm25 = BM25Okapi([c.lower().split() for c in chunks])

    def search(self, query: str, k: int = 5, rerank: bool = False) ->
list[str]:
        pool = min(max(k * 3, 10), len(self.chunks))
        qv = _get_embedder().encode([query], normalize_embeddings=True)
        _, dense_ids = self.index.search(np.asarray(qv, dtype="float32"),
pool)
        sparse_ids = np.argsort(self.bm25.get_scores(query.lower().split()))
[:::-1][:pool]
        fused = rrf([list(dense_ids[0]), list(sparse_ids)])
        if not rerank:
            return [self.chunks[i] for i in fused[:k]]
        shortlist = fused[:pool]
        scores = _get_reranker().predict([(query, self.chunks[i]) for i in
shortlist])
        order = np.argsort(scores)[::-1][:k]
        return [self.chunks[shortlist[i]] for i in order]

```

5.6 app/quiz/llm.py : akela network wala file

Sirf yahi file Ollama se baat karti hai. Do tarah ki failure alag hain: **LLM unreachable** (exception, fallback chalega) aur **LLM ne kachra JSON diya** (khaali dict, validator reject karega).

```

import json

import requests

from app.config import settings

```

```

class LLMUnavailable(Exception):
    pass

class OllamaClient:
    def generate_json(self, prompt: str) -> dict:
        try:
            r = requests.post(
                f"{settings.ollama_url}/api/generate",
                json={
                    "model": settings.model,
                    "prompt": prompt,
                    "format": "json",
                    "stream": False,
                    "options": {"temperature": 0.3},
                },
                timeout=settings.llm_timeout,
            )
            r.raise_for_status()
            return json.loads(r.json()["response"])
        except (requests.RequestException, KeyError) as e:
            raise LLMUnavailable(str(e)) from e
        except ValueError: # bad JSON from the model
            return {}

```

5.7 app/quiz/validator.py : LLM ka output untrusted hai

`_norm` whitespace aur case ignore karta hai, taaki line-break ka fark grounding na tode.

```

from pydantic import ValidationError

from app.schemas import MCQ

def _norm(s: str) -> str:
    return " ".join(s.lower().split())

def is_grounded(mcq: MCQ, chunk: str) -> bool:
    return _norm(mcq.source_snippet) in _norm(chunk)

def validate(raw: dict, chunk: str) -> MCQ | None:
    try:
        mcq = MCQ(**raw)
    except (ValidationError, TypeError):
        return None
    return mcq if is_grounded(mcq, chunk) else None

```

5.8 app/quiz/generator.py: chunk se MCQ loop

`client` ko argument ki tarah lete hain (`OllamaClient` ya test ka `FakeLLM`). Ek chunk pe `max_retries` baar dobara try karta hai, phir aage badh jaata hai.

```

from app.quiz.validator import validate
from app.rag.chunker import pick_spread
from app.schemas import MCQ

PROMPT = """You are a quiz writer. Using ONLY the passage below, write ONE
multiple-choice question.
Return JSON with exactly these keys:
"question" (string),
"options" (list of 4 distinct strings),
"answer_index" (integer 0-3, the position of the correct option),
"source_snippet" (one exact sentence copied from the passage that proves the
answer).

Passage:
{chunk}
"""

def generate_quiz(chunks: list[str], client, n: int, max_retries: int = 1) ->
list[MCQ]:

```

```

questions: list[MCQ] = []
for chunk in pick_spread(chunks, n):
    for _ in range(max_retries + 1):
        mcq = validate(client.generate_json(PROMPT.format(chunk=chunk)),
chunk)
        if mcq:
            questions.append(mcq)
            break
return questions

```

5.9 app/quiz/fallback.py : SHA-256 keyed cache

Cache file ke andar `sha256` rakha hai. Upload ke bytes ka hash match nahi hua toh `None`, matlab kisi aur document pe cache kabhi nahi lagta.

```

import hashlib
import json
from pathlib import Path

from app.config import settings
from app.schemas import MCQ

def sha256_bytes(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

def load_cache() -> dict | None:
    p = Path(settings.demo_quiz_path)
    return json.loads(p.read_text()) if p.exists() else None

def cached_quiz_for(data: bytes, n: int) -> list[MCQ] | None:
    cache = load_cache()
    if cache and cache["sha256"] == sha256_bytes(data):
        return [MCQ(**q) for q in cache["questions"]][:n]
    return None

```

5.10 app/main.py : patla wiring layer

Flow: validate upload, (optional) cache shortcut, parse, chunk, (optional) topic search, generate. Live se kuch na mile toh cache dekho, warna **503** (LLM down) ya **502** (LLM ne kachra diya). `llm_client` module-level hai taaki tests use `monkeypatch` se badal sakein. Static mount sabse **last** mein, warna wo API routes ko dhak lega.

```

import os
import tempfile
from pathlib import Path

from fastapi import FastAPI, File, Form, HTTPException, UploadFile
from fastapi.staticfiles import StaticFiles

from app.config import settings
from app.parsers import PARSERS, parse_file
from app.quiz import fallback
from app.quiz.generator import generate_quiz
from app.quiz.llm import LLMUnavailable, OllamaClient
from app.rag.chunker import chunk_text
from app.schemas import QuizResponse

app = FastAPI(title="DocQuiz")
llm_client = OllamaClient()
STATIC = Path(__file__).parent / "static"

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/api/quiz/demo", response_model=QuizResponse)
def demo_quiz():
    cache = fallback.load_cache()
    if not cache:
        raise HTTPException(404, "No demo cache built yet")
    return QuizResponse(
        source="cached",
        filename=cache["filename"],
        questions=cache["questions"][: settings.max_questions],
    )

@app.post("/api/quiz", response_model=QuizResponse)
def make_quiz(file: UploadFile = File(...), topic: str | None = Form(None)):
    filename = file.filename or "upload"
    suffix = Path(filename).suffix.lower()
    if suffix not in PARSERS:
        raise HTTPException(400, f"Unsupported type '{suffix}'. Use: {'',
'.join(sorted(PARSERS))}")
    data = file.file.read()
    if len(data) > settings.max_upload_mb * 1024 * 1024:

```

```

        raise HTTPException(413, "File too large")
    n = settings.max_questions

    if settings.prefer_cache:
        cached = fallback.cached_quiz_for(data, n)
        if cached:
            return QuizResponse(source="cached", filename=filename,
questions=cached)

    with tempfile.NamedTemporaryFile(suffix=suffix, delete=False) as tmp:
        tmp.write(data)
    try:
        text = parse_file(tmp.name)
    except ValueError as e:
        raise HTTPException(422, str(e))
    finally:
        os.unlink(tmp.name)

    chunks = chunk_text(text, settings.chunk_size, settings.chunk_overlap)
    if topic:
        from app.rag.retriever import HybridRetriever # heavy import, only
when needed

        chunks = HybridRetriever(chunks).search(topic, k=n, rerank=True)

    questions, llm_down = [], False
    try:
        questions = generate_quiz(chunks, llm_client, n)
    except LLMUnavailable:
        llm_down = True

    if questions:
        return QuizResponse(source="live", filename=filename,
questions=questions)
    cached = fallback.cached_quiz_for(data, n)
    if cached:
        return QuizResponse(source="cached", filename=filename,
questions=cached)
    if llm_down:
        raise HTTPException(503, "Quiz model unreachable. Is Ollama
running?")
    raise HTTPException(502, "Model returned no valid questions. Try again.")

app.mount("/", StaticFiles(directory=STATIC, html=True), name="static")

```

5.11 app/static/index.html : simple quiz UI

Sab `textContent` se, `innerHTML` se nahi (XSS). Badge batata hai quiz live hai ya cached. Flashcard mode optional extra hai: ek button jo option hide karke answer reveal kare.

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8"><title>DocQuiz</title>
<style>
  body{font-family:system-ui;max-width:720px;margin:2rem auto;padding:0 1rem}
  .card{border:2px solid #222;border-radius:10px;padding:1rem;margin:1rem
0;box-shadow:4px 4px 0 #222}
  .opt{display:block;width:100%;text-align:left;margin:.3rem
0;padding:.5rem;border:1px solid #888;border-
radius:6px;background:#fff;cursor:pointer}
  .ok{background:#c8f7c5}.bad{background:#f7c5c5}
  .badge{font-size:.8rem;padding:2px 8px;border-radius:99px;background:#eee}
</style>
</head>
<body>
<h1>DocQuiz</h1>
<input type="file" id="f" accept=".pdf,.docx,.pptx,.txt,.md">
<input type="text" id="topic" placeholder="optional topic">
<button id="go">Make quiz</button>
<button id="demo">Load demo quiz</button>
<p><span id="status"></span></p>
<div id="quiz"></div>
<h2 id="score"></h2>
<script>
const $ = id => document.getElementById(id);
let score = 0, answered = 0, total = 0;

function render(data) {
  const box = $("quiz"); box.innerHTML = ""; $("score").textContent = "";
  score = 0; answered = 0; total = data.questions.length;
  $("status").className = "badge";
  $("status").textContent = data.source === "cached" ? "cached (pre-
generated)" : "generated live";
  data.questions.forEach((q, i) => {
    const card = document.createElement("div"); card.className = "card";
    const h = document.createElement("h3"); h.textContent = `${i + 1}.
${q.question}`; card.append(h);
    q.options.forEach((opt, j) => {
      const b = document.createElement("button"); b.className = "opt";
      b.textContent = opt;
```

```

        b.onclick = () => {
            if (card.dataset.done) return;
            card.dataset.done = 1; answered++;
            if (j === q.answer_index) { b.classList.add("ok"); score++; }
            else { b.classList.add("bad"); card.children[q.answer_index +
1].classList.add("ok"); }
            if (answered === total) $("score").textContent = `Score:
${score}/${total}`;
        };
        card.append(b);
    });
    box.append(card);
});
}

async function call(url, opts) {
    $("status").className = ""; $("status").textContent = "Working... (local
LLM can take a minute)";
    const r = await fetch(url, opts);
    if (!r.ok) { $("status").textContent = (await r.json()).detail || "Error";
return; }
    render(await r.json());
}

$("go").onclick = () => {
    const f = $("f").files[0]; if (!f) return;
    const fd = new FormData(); fd.append("file", f);
    if ($("topic").value.trim()) fd.append("topic", $("topic").value.trim());
    call("/api/quiz", { method: "POST", body: fd });
};
$("demo").onclick = () => call("/api/quiz/demo");
</script>
</body>
</html>

```

5.12 scripts/build_demo_cache.py : cache ek baar banao

Local Ollama chalu rakho, `python -m scripts.build_demo_cache` chalao. Phir `data/demo/demo_quiz.json` **khud padho**: har answer sahi hai ya nahi, galat wale hata do. Sirf reviewed file commit karo.

```

import json
from pathlib import Path

from app.config import settings
from app.parsers import parse_file

```

```

from app.quiz.fallback import sha256_bytes
from app.quiz.generator import generate_quiz
from app.quiz.llm import OllamaClient
from app.rag.chunker import chunk_text

PDF = Path("data/demo/demo_doc.pdf")

chunks = chunk_text(parse_file(str(PDF)), settings.chunk_size,
settings.chunk_overlap)
questions = generate_quiz(chunks, OllamaClient(), n=8)
out = {
    "sha256": sha256_bytes(PDF.read_bytes()),
    "filename": PDF.name,
    "questions": [q.model_dump() for q in questions],
}
Path(settings.demo_quiz_path).write_text(json.dumps(out, indent=2))
print(f"Saved {len(questions)} questions. Now REVIEW them by hand before
committing.")

```

Note: local LLM pe 5 questions mein ~1 se 2 minute lag sakta hai (har chunk ek alag call). Isliye default 5 rakha hai, aur isi liye cache demo ke liye useful hai.

❑ 6. Tests (pipeline ka asli gatekeeper)

Teen layers: **unit** (ek function), **API** (route end-to-end, LLM fake), **quality gate** (retrieval quality ek golden document pe). LLM ko kabhi real call nahi karte, kyunki wo slow, flaky aur non-deterministic hai. Uski jagah **FakeLLM** aata hai, aur ye sirf isliye possible hai ki **generator** ko client argument mein mila tha.

Config files

```

# pytest.ini
[pytest]
pythonpath = .
testpaths = tests

```

```

# requirements-dev.txt
pytest
pytest-cov
httpx
ruff

```

6.1 tests/samples.py:shared fakes

```
from app.quiz.llm import LLMUnavailable

CHUNK = "Photosynthesis converts light energy into chemical energy. It
happens in chloroplasts."

GOOD = {
    "question": "Where does photosynthesis happen?",
    "options": ["Chloroplasts", "Nucleus", "Ribosome", "Vacuole"],
    "answer_index": 0,
    "source_snippet": "It happens in chloroplasts.",
}

class FakeLLM:
    """Returns pre-set answers, one per call. No network."""

    def __init__(self, responses):
        self.responses = list(responses)
        self.calls = 0

    def generate_json(self, prompt):
        self.calls += 1
        return self.responses.pop(0) if self.responses else {}

class DownLLM:
    def generate_json(self, prompt):
        raise LLMUnavailable("ollama is down")
```

6.2 tests/test_chunker.py

```
from app.rag.chunker import chunk_text, pick_spread

def test_chunks_respect_size():
    text = "\n\n".join(["word " * 50] * 20)
    chunks = chunk_text(text, size=300, overlap=30)
    assert len(chunks) > 1
    assert all(len(c) <= 300 for c in chunks)

def test_huge_paragraph_is_split():
    assert len(chunk_text("a" * 2000, size=500, overlap=50)) >= 4

def test_pick_spread_covers_whole_document():
    picked = pick_spread([str(i) for i in range(100)], 5)
    assert picked[0] == "0" and int(picked[-1]) >= 70
```

6.3 tests/test_parsers.py

DOCX/PPTX test ke andar hi bana lete hain, koi binary fixture commit nahi karni padti. PDF ke liye demo PDF use hota hai, jisse ye bhi pakda jaata hai ki demo file parse hoti hai.

```
import pytest
from docx import Document
from pptx import Presentation

from app.parsers import parse_file

def test_txt(tmp_path):
    p = tmp_path / "a.txt"
    p.write_text("hello world")
    assert parse_file(str(p)) == "hello world"

def test_docx(tmp_path):
    p = tmp_path / "a.docx"
    d = Document()
    d.add_paragraph("Cells have a nucleus.")
    d.save(str(p))
    assert "nucleus" in parse_file(str(p))
```

```

def test_pptx(tmp_path):
    p = tmp_path / "a.pptx"
    prs = Presentation()
    slide = prs.slides.add_slide(prs.slide_layouts[1])
    slide.shapes.title.text = "Mitochondria"
    prs.save(str(p))
    assert "Mitochondria" in parse_file(str(p))

def test_demo_pdf_is_parseable():
    assert len(parse_file("data/demo/demo_doc.pdf")) > 100

def test_unsupported_extension(tmp_path):
    p = tmp_path / "a.exe"
    p.write_bytes(b"x")
    with pytest.raises(ValueError):
        parse_file(str(p))

def test_empty_file_rejected(tmp_path):
    p = tmp_path / "a.txt"
    p.write_text(" ")
    with pytest.raises(ValueError):
        parse_file(str(p))

```

6.4 tests/test_validator.py

Har rejection rule ka ek test: yahi tumhara "LLM untrusted hai" ka saboot hai.

```

from app.quiz.validator import validate
from tests.samples import CHUNK, GOOD

def test_valid_question_passes():
    assert validate(GOOD, CHUNK) is not None

def test_rejects_bad_answer_index():
    assert validate(**GOOD, "answer_index": 7, CHUNK) is None

def test_rejects_duplicate_options():
    assert validate(**GOOD, "options": ["A", "A", "B", "C"], CHUNK) is None

```

```
def test_rejects_ungrounded_snippet():
    assert validate(**GOOD, "source_snippet": "Mitochondria make ATP."),
    CHUNK) is None

def test_rejects_missing_keys():
    assert validate({}, CHUNK) is None
```

6.5 tests/test_generator.py

```
from app.quiz.generator import generate_quiz
from tests.samples import CHUNK, GOOD, FakeLLM

OTHER_CHUNK = "Water boils at 100 degrees Celsius at sea level."
OTHER_GOOD = {
    "question": "At what temperature does water boil at sea level?",
    "options": ["100 C", "50 C", "0 C", "150 C"],
    "answer_index": 0,
    "source_snippet": "Water boils at 100 degrees Celsius at sea level.",
}

def test_one_question_per_chunk():
    llm = FakeLLM([GOOD, OTHER_GOOD])
    qs = generate_quiz([CHUNK, OTHER_CHUNK], llm, n=2)
    assert len(qs) == 2

def test_retries_after_garbage():
    llm = FakeLLM([{}, GOOD])
    qs = generate_quiz([CHUNK], llm, n=1, max_retries=1)
    assert len(qs) == 1 and llm.calls == 2

def test_gives_up_after_retries():
    llm = FakeLLM([{}, {}, {}])
    assert generate_quiz([CHUNK], llm, n=1, max_retries=1) == []
    assert llm.calls == 2
```

6.6 tests/test_retriever.py

Ye real embedding model use karta hai (Docker image mein pehle se baked hai), isliye ye ek **integration** test hai.

```

from app.rag.retriever import HybridRetriever, rrf

CHUNKS = [
    "Photosynthesis happens in chloroplasts.",
    "The French Revolution began in 1789.",
    "Water boils at 100 degrees Celsius.",
]

def test_rrf_prefers_items_ranked_high_in_both_lists():
    assert rrf([[1, 2, 3], [1, 3, 2]])[0] == 1

def test_hybrid_search_finds_relevant_chunk():
    r = HybridRetriever(CHUNKS)
    assert "chloroplasts" in r.search("where does photosynthesis occur", k=1)
[0]

def test_rerank_path_runs():
    r = HybridRetriever(CHUNKS)
    assert "1789" in r.search("when did the French Revolution start", k=1,
rerank=True)[0]

```

6.7 tests/test_api.py: route-level + fallback behaviour

Yahan fallback ka poora contract test hota hai: LLM down + known file = cached, LLM down + koi aur file = 503, aur cache kabhi dusre document pe nahi lagta.

```

import dataclasses
import json

from fastapi.testclient import TestClient

from app import main as main_module
from app.quiz import fallback
from tests.samples import CHUNK, GOOD, DownLLM, FakeLLM

client = TestClient(main_module.app)

def _use_cache(monkeypatch, tmp_path, data: bytes):
    cache = {"sha256": fallback.sha256_bytes(data), "filename": "demo.txt",
"questions": [GOOD]}
    p = tmp_path / "cache.json"

```

```

    p.write_text(json.dumps(cache))
    monkeypatch.setattr(
        fallback, "settings", dataclasses.replace(fallback.settings,
demo_quiz_path=str(p))
    )

def test_health():
    assert client.get("/health").json() == {"status": "ok"}

def test_rejects_unsupported_type():
    r = client.post("/api/quiz", files={"file": ("x.exe", b"abc")})
    assert r.status_code == 400

def test_live_quiz(monkeypatch):
    monkeypatch.setattr(main_module, "llm_client", FakeLLM([GOOD]))
    r = client.post("/api/quiz", files={"file": ("n.txt", CHUNK.encode())})
    assert r.status_code == 200
    assert r.json()["source"] == "live"

def test_llm_down_without_cache_is_503(monkeypatch, tmp_path):
    missing = str(tmp_path / "none.json")
    monkeypatch.setattr(
        fallback, "settings", dataclasses.replace(fallback.settings,
demo_quiz_path=missing)
    )
    monkeypatch.setattr(main_module, "llm_client", DownLLM())
    r = client.post("/api/quiz", files={"file": ("n.txt", CHUNK.encode())})
    assert r.status_code == 503

def test_llm_down_serves_cache_for_known_file(monkeypatch, tmp_path):
    data = CHUNK.encode()
    _use_cache(monkeypatch, tmp_path, data)
    monkeypatch.setattr(main_module, "llm_client", DownLLM())
    r = client.post("/api/quiz", files={"file": ("demo.txt", data)})
    assert r.status_code == 200
    assert r.json()["source"] == "cached"

def test_cache_never_served_for_a_different_file(monkeypatch, tmp_path):
    _use_cache(monkeypatch, tmp_path, b"the demo file")
    monkeypatch.setattr(main_module, "llm_client", DownLLM())

```

```

    r = client.post("/api/quiz", files={"file": ("other.txt",
CHUNK.encode())})
    assert r.status_code == 503

def test_demo_endpoint(monkeypatch, tmp_path):
    _use_cache(monkeypatch, tmp_path, b"x")
    r = client.get("/api/quiz/demo")
    assert r.status_code == 200 and r.json()["source"] == "cached"

```

6.8 Quality gate: tests/golden/ + scripts/quality_gate.py

Ek chhota document (notes.txt , ~1 page, apne subject ke notes) aur 6 se 8 sawal jinka jawab us document mein kisi ek jagah likha ho. Gate check karta hai ki **top-3 retrieved chunks mein sahi jagah mili ya nahi** (hit-rate@3). Chunking ya embedding ka koi bhi change retrieval ko bigaade toh build fail.

```

{
  "doc": "tests/golden/notes.txt",
  "cases": [
    {"query": "where does photosynthesis happen", "must_contain":
"chloroplast"},
    {"query": "what is the capital of France", "must_contain": "Paris"}
  ]
}

```

must_contain wo chhota phrase hai jo sahi chunk mein hona hi chahiye. Apne notes.txt ke hisaab se likho.

```

import json
import sys
from pathlib import Path

from app.rag.chunker import chunk_text
from app.rag.retriever import HybridRetriever

def main(threshold: float = 0.8) -> int:
    golden = json.loads(Path("tests/golden/golden.json").read_text())
    text = Path(golden["doc"]).read_text()
    retriever = HybridRetriever(chunk_text(text, 400, 50))
    hits = 0
    for case in golden["cases"]:
        top = retriever.search(case["query"], k=3)
        if any(case["must_contain"].lower() in chunk.lower() for chunk in

```

```

top):
    hits += 1
    rate = hits / len(golden["cases"])
    print(f"retrieval hit-rate@3 = {rate:.2f} (threshold {threshold})")
    return 0 if rate >= threshold else 1

if __name__ == "__main__":
    sys.exit(main())

```

Chalane ka tareeka: `python -m scripts.quality_gate`. Exit code 1 aaya toh Jenkins stage fail.

Coverage gate: `pytest --cov=app --cov-fail-under=80`. 80% se kam coverage = build fail.

□ 7. Docker: image, stages, compose

7.1 Multi-stage Dockerfile kyun?

Ek Dockerfile mein 3 stage: **base** (runtime deps + models), **test** (base + pytest + tests), **runtime** (base + non-root user). Jenkins `test` stage pe pytest chalata hai, phir `runtime` ko ship karta hai. Faayde: (1) test tools production image mein nahi jaate, (2) image chhoti, (3) Trivy sirf us image ko scan karta hai jo actually deploy hogi, (4) **wahi base layers** pe test hota hai jo ship hota hai.

Teen cheezein jo RAG image ko bhaari ya fragile banati hain, aur unka fix:

- `sentence-transformers` torch kheench laata hai (GBs). Fix: **CPU-only torch** pehle install karo.
- Models runtime pe download hoon toh container ka pehla start slow aur network pe dependent. Fix: **build time pe download** karke image mein bake karo.
- Root se chalna security risk hai. Fix: `appuser` banao. Isiliye `HF_HOME` alag folder mein rakhte hain aur read permission dete hain, warna non-root user models nahi padh payega.

```

# ----- base: runtime deps + baked models -----
FROM python:3.11-slim AS base
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
HF_HOME=/opt/hf
WORKDIR /srv

# requirements pehle copy: code badalne pe ye layer cache se aayegi
COPY requirements.txt .

```

```

RUN pip install torch --index-url https://download.pytorch.org/whl/cpu \
  && pip install -r requirements.txt

RUN python -c "from sentence_transformers import SentenceTransformer,
CrossEncoder; SentenceTransformer('all-MiniLM-L6-v2'); CrossEncoder('cross-
encoder/ms-marco-MiniLM-L-6-v2')" \
  && chmod -R a+rX /opt/hf

COPY app ./app
COPY data ./data

# ----- test: ship nahi hota -----
FROM base AS test
COPY requirements-dev.txt pytest.ini ./
RUN pip install -r requirements-dev.txt
COPY scripts ./scripts
COPY tests ./tests
CMD ["pytest", "--cov=app", "--cov-fail-under=80", "-q"]

# ----- runtime: ye ship hota hai -----
FROM base AS runtime
RUN useradd -m appuser
USER appuser
EXPOSE 8000
HEALTHCHECK --interval=10s --timeout=3s --retries=5 \
  CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Layer caching (viva favourite): Docker har instruction ek layer banata hai aur tab tak reuse karta hai jab tak uska input nahi badla. `requirements.txt` pehle copy karke install kiya, code baad mein. Code badalne pe sirf `COPY app` se neeche ki layers dobara banti hain, heavy pip install nahi.

HEALTHCHECK : container khud batata hai ki wo healthy hai ya nahi. `python:slim` mein `curl` nahi hota, isliye Python se hi check. Jenkins ka smoke test isi status ko padhta hai.

```
# requirements.txt (jab sab chal jaye: pip freeze se versions pin karo)
fastapi
uvicorn
python-multipart
pydantic>=2
requests
pypdf
python-docx
python-pptx
sentence-transformers
faiss-cpu
rank-bm25
numpy
```

`python-multipart` ke bina file upload route start hi nahi hoga. Ye common gotcha hai.

```
# .dockerignore
.git
.venv
__pycache__
*.pyc
.pytest_cache
.coverage
.env
```

7.2 `docker-compose.yml`

Compose = container ki config ek file mein (image, port, env). Image ka naam aur tag env se aate hain, taaki Jenkins `IMAGE_TAG=<sha>` dekar specific version deploy kar sake, aur rollback mein purana tag.

Ollama kahan chalega? Tum Mac pe ho. Ollama native Mac pe GPU (Metal) use karta hai, Docker ke andar sirf CPU, aur bahut slow. Isliye default: **host ka Ollama** use karo (`host.docker.internal`). Docker wala Ollama sirf optional profile hai.

```

services:
  app:
    image: ${IMAGE:-shweta1275/docquiz}:${IMAGE_TAG:-latest}
    ports:
      - "8000:8000"
    environment:
      OLLAMA_BASE_URL: ${OLLAMA_BASE_URL:-http://host.docker.internal:11434}
      OLLAMA_MODEL: ${OLLAMA_MODEL:-llama3}
      PREFER_CACHE: ${PREFER_CACHE:-false}
    restart: unless-stopped

  ollama:
    # optional: docker compose --profile with-ollama up
    image: ollama/ollama:latest
    profiles: ["with-ollama"]
    volumes:
      - ollama_data:/root/.ollama

volumes:
  ollama_data:

```

Agar `with-ollama` profile use karo toh `OLLAMA_BASE_URL=http://ollama:11434` set karo aur ek baar `docker compose exec ollama ollama pull llama3` chalo.

7.3 Local pe step by step check (Jenkins se pehle)

```

docker build --target test -t docquiz:test .
docker run --rm docquiz:test # pytest + coverage
docker run --rm docquiz:test python -m scripts.quality_gate
docker build --target runtime -t docquiz:dev .
IMAGE=docquiz IMAGE_TAG=dev docker compose up -d
curl localhost:8000/health

```

Ye sab local pe green na ho toh Jenkins mein daalna time waste hai.

8. Jenkins pipeline + Trivy

8.1 Jenkins ko Docker ke saath kaise chalaye

Jenkins khud container mein chalega, lekin pipeline ko `docker build` bhi chalana hai. Solution: host ka Docker socket Jenkins container mein mount karo (**Docker-out-of-Docker**): Jenkins ke andar ka `docker` CLI host ke Docker daemon se baat karta hai. Matlab pipeline ke containers Jenkins ke **andar nahi**, host pe chalte hain (siblings, children nahi).

```
# jenkins/Dockerfile
FROM jenkins/jenkins:lts-jdk17
USER root
RUN apt-get update && apt-get install -y ca-certificates curl gnupg git \
  && install -m 0755 -d /etc/apt/keyrings \
  && curl -fsSL https://download.docker.com/linux/debian/gpg -o \
  /etc/apt/keyrings/docker.asc \
  && echo "deb [arch=$(dpkg --print-architecture) signed- \
  by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/debian $(. \
  /etc/os-release && echo $VERSION_CODENAME) stable" > \
  /etc/apt/sources.list.d/docker.list \
  && apt-get update && apt-get install -y docker-ce-cli docker-compose-plugin \
  && rm -rf /var/lib/apt/lists/*
```

```
docker build -t jenkins-docker ./jenkins
docker run -d --name jenkins -u root -p 8080:8080 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  jenkins-docker
```

`-u root` isliye ki socket ki permission ka jhanjhat local lab mein na ho. Real production mein ye nahi karte (socket access = host pe root jaisa), ye tradeoff viva mein khud bata dena. Pehli baar password: `docker logs jenkins`. Suggested plugins install karo.

Credentials: Manage Jenkins, Credentials, Global, Add, *Username with password*. ID = `dockerhub`, username = Docker Hub username, password = **Access Token** (Docker Hub, Account settings, Security). Password kabhi nahi.

Job: New Item, Pipeline, "Pipeline script from SCM", Git, tumhari repo URL, branch `main`, script path `Jenkinsfile`. Build Triggers mein **Poll SCM** `H/2 * * * *` (har ~2 min mein check). Webhook ke liye ngrok chahiye, wo baad ka optional extra hai.

8.2 Jenkinsfile stage by stage

```
pipeline {
  agent any
  options { disableConcurrentBuilds() }
  environment {
    IMAGE      = 'shweta1275/docquiz'           // <dockerhub-user>/docquiz
    DOCKERHUB  = credentials('dockerhub')      // creates DOCKERHUB_USR /
    DOCKERHUB_PSW
    LAST_GOOD  = "${JENKINS_HOME}/docquiz_last_good"
  }
  stages {
```

```

stage('Checkout') {
  steps {
    checkout scm
    script { env.TAG = sh(script: 'git rev-parse --short HEAD',
returnStdout: true).trim() }
  }
}
stage('Build test image') {
  steps { sh 'docker build --target test -t $IMAGE:test-$TAG .' }
}
stage('Lint') {
  steps { sh 'docker run --rm $IMAGE:test-$TAG ruff check app tests
scripts' }
}
stage('Unit + API tests') {
  steps { sh 'docker run --rm $IMAGE:test-$TAG' }
}
stage('Quality gate') {
  steps { sh 'docker run --rm $IMAGE:test-$TAG python -m
scripts.quality_gate' }
}
stage('Build runtime image') {
  steps { sh 'docker build --target runtime -t $IMAGE:$TAG -t
$IMAGE:latest .' }
}
stage('Trivy scan') {
  steps {
    sh '''
      docker run --rm \
        -v /var/run/docker.sock:/var/run/docker.sock \
        -v trivy-cache:/root/.cache/ \
        aquasec/trivy:latest image \
        --exit-code 1 --severity HIGH,CRITICAL --ignore-unfixed \
        $IMAGE:$TAG
    '''
  }
}
stage('Push') {
  steps {
    sh '''
      echo "$DOCKERHUB_PSW" | docker login -u "$DOCKERHUB_USR" --
password-stdin
      docker push $IMAGE:$TAG
      docker push $IMAGE:latest
    '''
  }
}

```

```

}
stage('Deploy') {
  steps { sh 'IMAGE_TAG=$TAG docker compose -p docquiz up -d' }
}
stage('Smoke test') {
  steps {
    sh '''
      CID=$(docker compose -p docquiz ps -q app)
      for i in $(seq 1 20); do
        STATUS=$(docker inspect -f '{{.State.Health.Status}}' $CID)
        [ "$STATUS" = "healthy" ] && break
        sleep 3
      done
      [ "$STATUS" = "healthy" ] || { echo "App never became healthy";
exit 1; }
      docker compose -p docquiz exec -T app python -c "import
urllib.request;
urllib.request.urlopen('http://localhost:8000/api/quiz/demo')"
    '''
  }
}
post {
  success { sh 'echo $TAG > $LAST_GOOD' }
  failure {
    sh '''
      if [ -f "$LAST_GOOD" ]; then
        echo "Rolling back to $(cat $LAST_GOOD)"
        IMAGE_TAG=$(cat $LAST_GOOD) docker compose -p docquiz up -d
      else
        echo "No previous good version to roll back to"
      fi
    '''
  }
  always { sh 'docker image prune -f || true' }
}
}

```

8.3 Har stage ka reasoning

Stage	Kya check/karta hai	Fail hone par
Build test image	<code>test</code> target build, models baked	Dockerfile ya deps toota

Stage	Kya check/karta hai	Fail hone par
Lint	ruff: unused imports, syntax issues	Code quality
Unit + API tests	pytest + coverage 80% gate	Logic toota ya coverage kam
Quality gate	retrieval hit-rate@3 >= 0.8	Chunking/embedding ne retrieval bigada
Build runtime image	Final non-root image	Build error
Trivy scan	HIGH/CRITICAL fixable CVEs	Vulnerable dependency ya base image
Push	Docker Hub pe :sha aur :latest	Login/token galat
Deploy	Compose naye tag se up	Image pull ya config error
Smoke test	Container healthy + /api/quiz/demo 200	Rollback chalta hai

SHA tag + rollback: har build ka image `:<git-sha>` se tag hota hai, toh pata rehta hai kaunsa code kaunsi image mein hai. Deploy hamesha SHA se hota hai, `latest` sirf convenience hai. Smoke test pass hua toh `docquiz_last_good` mein SHA likha jaata hai. Fail hua toh `post { failure }` usi purane SHA ko dobara deploy karta hai. Agar fail deploy se pehle hua (jaise test), toh wahi chalti hui good version dobara `up` hoti hai, koi nuksaan nahi.

Credentials: `credentials('dockerhub')` Jenkins ke secret store se aata hai aur logs mein `****` ban jaata hai. Shell scripts ko single quotes mein rakha hai taaki secret Groovy mein interpolate hoke log mein na dikhe.

8.4 Trivy ke baare mein

Trivy image ke OS packages aur Python packages ko known CVE database se match karta hai. `--severity HIGH,CRITICAL` sirf serious wale pe fail karta hai. `--ignore-unfixed` un CVEs ko ignore karta hai jinka abhi koi patch nahi hai (unpe tum kuch kar hi nahi sakte). `trivy-cache` volume isliye ki har run mein CVE database dobara download na ho.

Agar pehli run mein Trivy fail ho jaye (torch ya base image ki wajah se), **flag mat hatao**. Pehle base image tag update karo ya vulnerable package ka version badlo. Koi finding sach mein na-theek-hone-layak ho toh `.trivyignore` mein CVE ID likho **reason ke saath**. Demo mein ek known-vulnerable package version pin karke Trivy ka red stage dikhana achha impression deta hai.

8.5 Jenkins ke 4 gotchas (sab pe mera dhyan raha)

1. **Volume mount trap:** pipeline mein `docker run -v $PWD:/src ...` mat likho. `$PWD` Jenkins container ka path hai, wo host pe exist nahi karta, toh mount khaali aata hai. Isi wajah se lint/tests image ke **andar** chalte hain, workspace mount nahi hota.
2. Jenkins image mein `docker-compose-plugin` chahiye, warna `docker compose` command nahi milega (upar ke Dockerfile mein hai).
3. Docker Hub free tier pe pull rate limit hai. Base images ek baar pull hone ke baad cache mein rahenge.
4. Pehla build bahut slow hoga (torch + models). Doosra layer cache se aayega, isliye pehle build ke liye samay rakho.

9. Build order, fallbacks aur demo

Golden rule: ek time pe ek phase, har phase ke end pe green. Kisi bhi phase pe ruk jao, tab bhi submit karne layak cheez ho.

#	Phase	Pehle samjho (section)	Done ka matlab
0	Purane quiz app pe Jenkins pipeline green (fallback)	8	Push pe stages green, Docker Hub pe image
1	Repo + skeleton: folders, venv, <code>config.py</code> , <code>schemas.py</code>	3, 5.1, 5.2	<code>python -c "import app.schemas"</code> chale
2	Parsers + chunker + unke tests	4.1, 4.2, 5.3, 5.4, 6.2, 6.3	<code>pytest</code> green
3	LLM client + validator + generator + FakeLLM tests	4.5, 4.6, 5.6 to 5.8, 6.4, 6.5	Fake LLM ke saath tests green
4	API + UI + demo cache	4.7, 5.9 to 5.12, 6.7	Browser mein real Ollama se quiz, cache badge dikhe
5	Local Docker: 3 stages + compose	7	<code>docker run docquiz:test</code> green, <code>/health</code> chale
6	Jenkins: sirf Checkout + Lint + Tests	8.1, 8.2	Push pe tests Jenkins mein chalein
7	Gates + build + Trivy + Push, ek-ek stage add karke	8.3, 8.4	Docker Hub pe <code>:sha</code> image
8	Deploy + Smoke test + Rollback	8.3	Pipeline se app live, fail pe purana version wapas

#	Phase	Pehle samjho (section)	Done ka matlab
9	Retriever + topic mode + quality gate (optional)	4.4, 5.5, 6.6, 6.8	Topic dalke focused quiz, gate green
10	README + architecture diagram + screenshots + demo rehearsal	2	Fresh clone se sab chale

Time kam pade toh kaato (is order mein): phase 9 (topic mode), phir quality gate, phir rollback. Trivy, tests, aur coverage gate **mat** kaato, wahi rubric aur resume ka core hain.

Har phase ka tareeka (aur Claude Code ke saath bhi yahi): pehle section padho aur khud ek line mein samjhao "ye kyun hai", phir code chalo, phir ek cheez jaanbujhkar todke dekho (jaise ek test fail karo, answer_index 9 karo) ki system kaise react karta hai. Todke dekhne se hi samajh aata hai.

Demo script (5 minute)

1. Architecture diagram dikhao (30 sec): "upload se quiz, aur peeche CI/CD."
2. Browser: demo PDF upload karo, live quiz generate hone do. Badge "**generated live**" dikhega. Ek question ka `source_snippet` dikhao ki ye document se aaya.
3. Code mein ek chhota change karo, push karo. Jenkins stages ek-ek karke green hote dikhao.
4. **Jaanbujhkar ek test fail karo**, push karo, pipeline lal hokar ruk jaaye aur deploy na ho. Phir wapas theek karo.
5. Docker Hub pe SHA-tagged images dikhao, aur Trivy ka scan output.
6. Safety net bolo: "LLM down ho toh known demo file pe cache serve hota hai aur UI batata hai." Ollama band karke dikha bhi sakti ho.

Demo ke din ka checklist: Ollama chalu aur model pulled, Jenkins container up, Docker Desktop up, Docker Hub token valid, demo PDF aur reviewed `demo_quiz.json` repo mein, pipeline ek baar fresh clone se chal chuki ho, internet ka plan B (hotspot).

README mein kya ho

Project ka ek-line summary, architecture diagram, tech stack, local run steps, pipeline stages ki table, screenshots (Jenkins green, Docker Hub, Trivy), limitations (scanned PDF, LLM accuracy, no monitoring), aur future work (Prometheus `/metrics`, Kubernetes, OCR). Limitations khud likhna maturity dikhata hai.

10. Viva Q&A (jawab apne shabdon mein bolna)

Sawal	Chhota jawab
CI aur CD mein fark?	CI: har push pe automatically build + test. CD: test pass hone ke baad artifact (image) automatically deploy.
Pipeline-as-code kyun?	<code>Jenkinsfile</code> repo mein hai, toh version-controlled, review ho sakti hai, aur Jenkins UI kharab ho tab bhi wapas ban jaati hai.
Image vs container?	Image read-only blueprint (layers). Container uska running instance.
Multi-stage build kyun?	Test tools production image mein na jayein, image chhoti ho, aur Trivy sirf shipped image scan kare.
Layer caching kaise kaam karta hai?	Har instruction ek layer. Input na badla toh cache. Isliye requirements pehle copy, code baad mein.
<code>latest</code> ki jagah SHA tag kyun?	<code>latest</code> badalta rehta hai. SHA se pata hai kaunsa code kaunsi image mein hai, aur rollback exact version pe ho sakta hai.
Secrets kaise secure rakhe?	Docker Hub Access Token Jenkins credentials store mein, <code>credentials()</code> se inject, logs mein masked. Code ya repo mein kabhi nahi.
Pipeline fail ho toh?	Stage wahin ruk jaata hai, aage ke stages (push, deploy) nahi chalte. Deploy ke baad fail ho toh <code>post failure</code> purana SHA wapas deploy karta hai.
Docker-out-of-Docker kya hai?	Jenkins container mein host ka Docker socket mount. Pipeline ke containers host pe siblings ki tarah chalte hain. Tradeoff: socket access host pe root jaisa hai, isliye sirf local lab mein.
Quiz generation mein retrieval kyun nahi?	Quiz mein query nahi hoti, poora document cover karna hota hai, isliye chunks barabar doori pe sample karte hain. Retrieval sirf topic mode mein.
Hybrid search kyun?	Dense semantic matlab pakadta hai, BM25 exact keywords. RRF dono ko rank se jodta hai. Cross-encoder reranker shortlist ko dobara accurate score karta hai.

Sawal	Chhota jawab
LLM output pe bharosa kyun nahi?	LLM galat format ya hallucination de sakta hai, isliye Pydantic schema + grounding check lagaya. Wo untrusted input jaisa hai.
Grounding check kya prove karta hai?	Sirf ki LLM ne jo snippet diya wo source chunk mein sach mein hai. Ye prove nahi karta ki <code>answer_index</code> sahi hai. Isliye demo cache haath se review hua.
Cache ka kya matlab? Kya ye fake hai?	Nahi. Real pipeline se generate hua, review hua, aur file ke SHA-256 se keyed hai. Sirf wahi file pe lagta hai, aur UI mein "cached" badge dikhta hai.
Production-ready hai?	Nahi poora. CI, tests, scan, versioned deploy, rollback hai. Monitoring, IaC, orchestration, secrets manager, auth nahi. Future work mein Prometheus aur Kubernetes.
Coverage gate ka matlab?	80% se kam code tests se chhua toh build fail. Ye guarantee nahi ki tests achhe hain, par ek floor hai.
Trivy kya karta hai?	Image ke OS aur Python packages ko known CVE database se match karta hai, HIGH/CRITICAL fixable pe pipeline fail.

11. Claude Code ke liye master prompt

Kaise use karna hai: (1) Is doc ko Markdown mein export karke repo mein `docs/PLAN.md` naam se rakho, taaki Claude Code ke paas poora code aur reasoning ho. (2) Naya khaali folder `docquiz/` banao, usme `git init` karo, Claude Code wahan kholo. (3) Neeche wala prompt paste karo. (4) Har phase ke baad Claude Code ruk jaayega, tum section padh ke samjho, tab "continue" bolo.

```
You are my pair-programming mentor and implementer. We are building
QuizItOut, a DevOps
mini-project: a web app that turns uploaded documents (pdf, docx, pptx, txt,
md) into MCQ
quizzes using a local LLM, wrapped in a full Jenkins CI/CD pipeline.
```

NAMING

```
- Project name: QuizItOut. docs/PLAN.md uses the working name "DocQuiz" in
places. Use
```

"quizitout" for the repo/folder name, the Docker image (<dockerhub-username>/quizitout), the

compose project name and the Jenkins job. Use "QuizItOut" in the UI title and README.

Keep the Python package name `app`.

- Ask me for my Docker Hub username before writing it anywhere (it may differ from GitHub).

SOURCE OF TRUTH

- docs/PLAN.md contains the complete plan, every file's code, tests, Dockerfile, compose,

Jenkinsfile and the reasoning. Read it fully before writing anything.

- Implement exactly what the plan says. If you think something in it is wrong or outdated

(library API, flag, version), tell me and propose a fix. Do not silently deviate.

- Do not add features, frameworks or files that are not in the plan.

TECH CONSTRAINTS

- Python 3.11, FastAPI, Pydantic v2, pypdf, python-docx, python-pptx, sentence-transformers

(CPU only), faiss-cpu, rank-bm25, requests. No LangChain or LlamaIndex.

- LLM = Ollama over HTTP. Tests must NEVER call a real LLM; use the FakeLLM from tests/samples.py.

- Frontend = one static HTML file, no build step, LLM text inserted with textContent only.

- Runtime Docker image: multi-stage (base/test/runtime), CPU-only torch, models baked in at

build time, non-root user, HEALTHCHECK.

- Jenkins runs in a container with the host Docker socket mounted (Docker-out-of-Docker).

Never use `-v \$PWD:...` bind mounts inside pipeline steps.

HONESTY RULES (important)

- The demo cache (data/demo/demo_quiz.json) is a pre-generated, human-reviewed cache keyed by

the file's SHA-256. It must only be served when the SHA matches, and every response must

carry source: "live" or "cached". Never present cached output as live generation.

- Never hardcode secrets. Docker Hub uses an access token stored in Jenkins credentials.

DIVISION OF WORK (very important)

YOU (Claude Code) do everything that lives in the repo or can run in this terminal: writing and

editing files, creating the .venv and installing requirements, pytest, ruff, docker build/run/compose, local curl checks, and LOCAL git commits (one commit per finished phase, message "phase N: <summary>").

I (the human) do everything outside the repo or needing my accounts/secrets:

- Installing software (Docker Desktop, Ollama, Git, Python) and pulling the LLM model.
- Creating the GitHub repo, authenticating git, and running `git push`.
- Docker Hub account, repository and access token.
- Everything inside the Jenkins web UI (unlock, plugins, credentials, creating the pipeline job, running builds).
- Adding teammates, taking screenshots, rehearsing the demo.

For every task on my side, do NOT just say "set up X". STOP and give me a numbered, click-by-click checklist: exact copy-pasteable commands, exact menu names, what success looks like, and the 2-3 most likely errors with fixes. Then wait. When I say "done", ask me to paste the output of ONE verification command and confirm it before moving on.

Never ask me to paste secrets (tokens, passwords) into this chat and never write them to files.

Never run `git push`, `docker push` or `docker login` yourself; I do those. Do not install system software yourself; tell me what is missing.

WORKING STYLE (I am learning, not just collecting code)

- Work in the numbered phases below. Before each phase, explain in plain language what we are about to build and why (architecture and reasoning first, max 10 lines).
- Then write the code, run the tests/commands for that phase, and show me the real output.
- After each phase, give me: (a) a 5-line summary of what each new file does, (b) one thing I should deliberately break to see how the system reacts, (c) two viva-style questions.
- Then STOP and wait for me to say "continue". Never start the next phase on your own.
- If a command fails, show the error, explain the cause, fix it, re-run. No hand-waving.

PHASES (marked HUMAN = I must do something before/during it)

0. PREFLIGHT. Run: git --version, docker --version, docker compose version,

```
python3 --version,
ollama --version, trivy --version (optional). Tell me exactly what is
missing.
HUMAN: install what is missing; create the empty GitHub repo `quizitout`;
run git init, add
the remote and authenticate. Give me those steps. (Trivy is NOT required
locally: the
pipeline runs it as the aquasec/trivy container.)
1. Skeleton: folder structure, .gitignore (include .venv, __pycache__, .env,
.coverage),
.dockerignore, requirements*.txt, pytest.ini, app/config.py,
app/schemas.py. Create .venv
and install requirements. Make the first local commit.
2. HUMAN: put a 2-3 page text-based PDF at data/demo/demo_doc.pdf. Then:
parsers + chunker
(app/parsers.py, app/rag/chunker.py) with tests/test_parsers.py,
tests/test_chunker.py.
3. LLM layer: app/quiz/llm.py, validator.py, generator.py, tests/samples.py,
test_validator.py,
test_generator.py.
4. API + UI + cache: app/quiz/fallback.py, app/main.py,
app/static/index.html,
scripts/build_demo_cache.py, tests/test_api.py.
HUMAN: make sure Ollama is running with the model pulled, run
build_demo_cache, review
data/demo/demo_quiz.json by hand, delete wrong questions. Give me these
steps.
5. Retriever (topic mode): app/rag/retriever.py, tests/test_retriever.py,
scripts/quality_gate.py,
tests/golden/golden.json skeleton. HUMAN: I write tests/golden/notes.txt
and the queries;
explain exactly what to write.
6. Docker: multi-stage Dockerfile, docker-compose.yml. Verify with: docker
build --target test,
docker run the test image, quality gate, build runtime image, compose up,
curl /health.
7. Jenkins: write jenkins/Dockerfile and the Jenkinsfile, building the
Jenkinsfile up ONE stage
at a time (Checkout, Lint, Tests first; then quality gate, runtime build,
Trivy, Push; then
Deploy, Smoke test, rollback). After each added stage: commit, tell me to
push and watch
Jenkins, and wait.
HUMAN: Docker Hub repo + access token; build and run the Jenkins
container; unlock, plugins,
credentials (id: dockerhub), create the Pipeline job, Poll SCM, run
```

builds. Give me click-by-click steps for each.

8. Docs: README.md (summary, diagram, run steps, pipeline table, limitations, future work) and a demo script. HUMAN: I take the screenshots; give me the exact list of screenshots needed.

DEFINITION OF DONE

- pytest passes with coverage $\geq 80\%$, ruff is clean, quality gate ≥ 0.8 .
- Runtime image passes Trivy (HIGH/CRITICAL, unfixed ignored) or every ignored CVE is in .trivyignore with a written reason.
- A push to main triggers Jenkins, which deploys by git-SHA tag, smoke-tests, and rolls back on failure.

Start by reading docs/PLAN.md, then summarize the architecture back to me in 10 lines to prove you understood, then begin Phase 0.

Tip: Claude Code ko ye prompt `CLAUDE.md` mein bhi rakh sakti ho, taaki har naye session mein wahi rules lagu rahein.

12. Setup checklist: Claude Code ka kaam vs tumhara kaam

Seedha rule: jo repo ke andar ya terminal mein ho jaye (code, tests, Docker build, local commit) wo Claude Code karega. Jo tumhare accounts, passwords ya browser UI se judha hai (GitHub, Docker Hub, Jenkins, Ollama) wo **tumhara** hai, aur Claude Code ko prompt mein bola hai ki wo har aise kaam pe ruk kar tumhe click-by-click steps de.

Trivy ke baare mein: tumhe kuch install nahi karna. Pipeline Trivy ko `aquasec/trivy` container ki tarah chalati hai, toh tumhare laptop pe Trivy ho ya na ho, farak nahi padta. Local CLI sirf tab kaam aata hai jab tum Jenkins se pehle khud `trivy image <naam>` try karna chaho.

Phase	Claude Code karega	Tum karogi
0 Preflight	Tools check karega: git, docker, python, ollama	Jo missing ho install, GitHub repo banana, git auth (A, B)
1 Skeleton	Folders, config, venv, pehla commit	<code>git push</code>
2 Parsers + chunker	Code + tests	Demo PDF <code>data/demo/</code> mein daalna (C)

Phase	Claude Code karega	Tum karogi
3 LLM layer	Validator, generator, FakeLLM tests	Kuch nahi
4 API + UI + cache	App, UI, cache script	Ollama chalu karke cache banana aur haath se review karna (D)
5 Retriever + gate	Retriever, gate script	<code>notes.txt</code> aur golden queries likhna (C)
6 Docker	Dockerfile, compose, local verify	Docker Desktop chalu rakhna
7 Jenkins	<code>Jenkinsfile</code> , <code>jenkins/Dockerfile</code>	Docker Hub token, Jenkins UI setup, builds chalana (E, F)
8 Docs	README, demo script	Screenshots, demo rehearsal (G)

A. Ek baar ka machine setup (Claude Code se pehle)

- ☐ **Docker Desktop** install karke kholo. Check: `docker --version`, `docker compose version`, `docker run hello-world`.
- ☐ **Git**: `git --version`. Phir `git config --global user.name "Shweta"` aur `git config --global user.email "<tumhari GitHub email>"`. Ye email GitHub account wali honi chahiye, warna commits tumhare naam pe nahi dikhenge.
- ☐ **Python 3.10+**: `python3 --version` (Docker image mein 3.11 use hoga, local pe koi bhi 3.10+ chalega).
- ☐ **Ollama** install. Check: `ollama --version`. Model: `ollama pull llama3` (~5 GB, wifi pe karo). Test: `ollama run llama3 "say hi"` aur `curl http://localhost:11434/api/tags` (model ka naam dikhna chahiye).
- ☐ **Claude Code** install karke ek khaali folder mein kholo.
- ☐ Disk space check: Docker images (torch + models) aur llama3 milake 10 GB se zyada lagta hai.

B. GitHub repo (Phase 0 mein)

- ☐ github.com, New repository. Name `quizitout`, **Public**, README/.gitignore/license mat tick karo (khaali repo chahiye).
- ☐ Settings, Collaborators, teammates ko add karo. Group project hai, toh sabke commits dikhne chahiye.
- ☐ Terminal mein: `mkdir quizitout && cd quizitout && git init -b main && git remote add origin https://github.com/shweta1275/quizitout.git`
- ☐ **Auth**: sabse aasan `gh auth login` (GitHub CLI) ya SSH key. HTTPS pe password nahi chalta, Personal Access Token chahiye.

- ☐ Pehla push (Phase 1 ke baad): `git push -u origin main`. Uske baad bas `git push`.
- ☐ Commit message style: `phase N: <kya kiya>`. Git history demo mein dikhegi, isliye saaf rakho.

C. Documents jo tumhe khud banane hain

- ☐ **Demo PDF** (Phase 2 se pehle): 2 se 3 page, apne kisi subject ke notes. Test: PDF mein text select aur copy ho sakta ho. Scanned image nahi chalegi. Save: `data/demo/demo_doc.pdf`.
- ☐ **Golden set** (Phase 5): `tests/golden/notes.txt` (~1 page notes) aur `golden.json` mein 6 se 8 sawal, har ek ke saath wo phrase jo jawab wale chunk mein hona chahiye. Claude Code format samjha dega, content tum likhogi.

D. Demo cache banana (Phase 4)

- ☐ Ollama chalu hai (`ollama serve` ya app khuli).
- ☐ `python -m scripts.build_demo_cache` chalao (1 se 3 minute).
- ☐ `data/demo/demo_quiz.json` kholo aur **har question padho**: sahi option par `answer_index` hai? Galat ya ajeeb question hata do. Kam se kam 5 achhe questions bachne chahiye.
- ☐ Commit karo. Ye file ka SHA demo PDF se bandha hai, toh **PDF badli toh cache dobara banana padega**.

E. Docker Hub (Phase 7 se pehle)

- ☐ `hub.docker.com` par account, phir Create repository `quizitout` (public).
- ☐ Account settings, Personal access tokens, Generate. Name `jenkins`, permission **Read & Write**. Token **ek hi baar** dikhta hai, copy karke password manager mein rakho.
- ☐ Apna **Docker Hub username** note karo. Claude Code wahi poochega aur `Jenkinsfile/compose` mein daalega.

F. Jenkins setup (Phase 7)

- ☐ Claude Code `jenkins/Dockerfile` dega. Phir: `docker build -t jenkins-docker ./jenkins` aur section 8.1 wala `docker run -d --name jenkins -u root -p 8080:8080 -v jenkins_home:/var/jenkins_home -v /var/run/docker.sock:/var/run/docker.sock jenkins-docker`.
- ☐ `docker logs jenkins` mein admin password milega. Browser mein `localhost:8080`, password daalo, **Install suggested plugins**, admin user banao.
- ☐ Verify: `docker exec jenkins docker ps` aur `docker exec jenkins docker compose version`. Dono chalne chahiye.

- ☐ Credentials: Manage Jenkins, Credentials, Global, Add Credentials, *Username with password*. Username = Docker Hub username, password = **access token**, ID = **dockerhub** (spelling exact, Jenkinsfile isi ID ko dhoondhta hai).
- ☐ New Item, naam **quizitout**, **Pipeline**. Definition: *Pipeline script from SCM*, SCM: Git, Repository URL: tumhari repo, Branch: ***/main**, Script Path: **Jenkinsfile**.
- ☐ Build Triggers mein **Poll SCM** **H/2 * * * * ***.
- ☐ **Build Now**. Console Output kholke dekho. Claude Code jab ek-ek stage add karega, har baar push karo aur green hone do, tab aage.
- ☐ Pehla Trivy run slow hoga (vulnerability database download), dhairya rakho.

Common errors:

Error	Matlab / fix
<code>permission denied ... docker.sock</code>	Jenkins container <code>-u root</code> ke bina chala. Container hata ke dobara chalo.
<code>docker: command not found</code> (Jenkins mein)	Image <code>jenkins/Dockerfile</code> se nahi bani, plain Jenkins image chal rahi hai.
<code>docker compose</code> not found	Dockerfile mein <code>docker-compose-plugin</code> ki line chhut gayi.
<code>unauthorized / denied</code> push par	Token galat, username galat, ya Credentials ID <code>dockerhub</code> se alag.
Port 8080 already in use	Doosra container ya app 8080 pe hai. Band karo ya <code>-p 8081:8080</code> use karo.
Jenkins git se clone nahi kar pa raha	Repo public hai? URL mein <code>.git</code> aur spelling check karo.

G. Antim hafte ki checklist (screenshots + rehearsal)

- ☐ Jenkins Stage View, sab green.
- ☐ Jenkins ka ek **jaanbujhkar fail kiya hua** run (lal stage, deploy skip).
- ☐ Docker Hub ka tags page (`:sha` aur `:latest`).
- ☐ Trivy ka console output.
- ☐ App ka "generated live" badge aur "cached" badge.
- ☐ GitHub commits page (sabke commits).
- ☐ **Fresh clone test:** `git clone <repo> /tmp/fresh`, wahan se compose chalake dekho ki sab chalta hai. Demo ke din wahi dikhana hai.

- ☐ Demo script ek baar poori speed se bolke practice karo (section 9).

Group split suggestion (3 log): ek Jenkins + Trivy, ek Docker + compose, ek tests + docs.
Teeno apne account se commit karein, taaki git history mein sabka kaam dikhe.